

Module 8 — Structured Workflows

Deciding how much of the control flow to hand to the model

Structured Workflows

Every agent you have built so far lets the model decide what happens next.

That is a choice, not a default, and usually the most expensive one available.

There is a spectrum from code-driven to model-driven. We will explore the spectrum and how to choose the right place for your workflow.

🎯 Learning goals

- Place a task on the **control-flow spectrum**: pipeline, router, plan-and-execute, ReAct.
- Build a **pipeline** where code owns every step, and a **router** with one auditable decision.
- Build a **ReAct loop** whose reasoning is visible, with guards that stop it running away.
- Build **plan-and-execute**, and exploit the fact that a plan is data.
- Run independent plan steps **concurrently**, and **pause a plan** for human approval.



Who owns the control flow?

Your code, or the model



Who owns the control flow?

How many decisions am I handing to the model? Every decision has a cost in time, tokens, and testability

The control-flow spectrum

1. Pipeline

Code decides every step

The model only generates text

Fully testable, cheapest, fastest

Breaks on anything unanticipated

2. Router

Model picks a branch, code runs it

One decision, and it is auditable

Still cheap and still testable

You must enumerate the branches

3. ReAct

Model decides every step

Adapts to whatever it finds

Most expensive, slowest, hardest to test

Can loop, wander, or never finish

4. Plan-and-execute

Model decides once, up front

Code executes the plan

The plan is data: show it, store it, resume it

A bad plan fails the whole run

The control-flow spectrum

Pipeline

```
# you wrote
# the order
a = check("reactor")
b = check("shields")
c = search_logs(q)

summarise([a, b, c])

# model calls: 1
# decisions:   0
```

Router

```
# model picks
# a branch you wrote
route = classify(task)

for step in BRANCHES[route]:
    run(step)

summarise(findings)

# model calls: 2
# decisions:   1
```

ReAct

```
# model decides
# every turn
while not done:
    s = decide(msgs)
    if s.final:
        break
    obs = run(s)
    msgs += obs

# decisions:   N
```

Plan-and-execute

```
# model wrote
# the order, once
plan = make_plan(goal)

for step in plan:
    run(step)

summarise(results)

# model calls: 2+
# decisions:   1
```

→ Push control flow as far left as the task allows

- Start at the **left** and move right only when the task actually breaks.
- Moving right buys **adaptability**. It costs tokens, latency, and testability.
- Most production "agents" are a pipeline or a router wearing a trench coat — and that is fine.
- Reach for ReAct when you genuinely **cannot know the steps** until you see the data.
- The spectrum is per-task, not per-product. One app can use all four.



Demo — one task, four strategies

Switch to terminal: python demo/01_control_flow.py



Pipeline

The fixed chain. Your code owns the order.

Code decides every step

- You write the sequence. The model only generates text where you ask.
- Usually that is **one call, at the end**, to summarise findings the code already gathered.
- The same request always takes the same path, so a unit test can lock it.
- Cheapest, fastest, and the right answer more often than people want to admit.

A pipeline is an ordinary function

```
findings = []
for name, args in [
    ("check_system", {"system": "reactor"}),
    ("check_system", {"system": "shields"}),
    ("search_logs", {"query": "plasma conduit"}),
]:
    findings.append(run_tool(name, args))

answer = synthesise(task, findings)   # the only model call
```

- There is no loop the model can get stuck in. Adding a step is adding a line.
- `synthesise` does not choose what runs. It only writes the sentence at the end.

It breaks when the path is not fixed

- A pipeline runs **every** step, including the ones this request did not need.
- It cannot look at the logs and decide the reactor needs a second check.
- It cannot grow a step you did not write.
- When requests come in a few known shapes, do not jump to ReAct. Add a router.



Router

The model picks a branch. Code runs it.

Agent chooses a branch

- The model does not choose tools. It chooses a **name from a list you wrote**.
- Each name points at a pipeline. After the choice, control returns to code.
- The route choice is the only thing you cannot unit test.
- Everything after the route choice is testable.

Classify, then run the branch

```
class Route(BaseModel):
    branch: Literal["systems", "logs", "full"]

route = client.responses.parse(
    model=MODEL, text_format=Route, input=task,
).output_parsed

branch = route.branch if route.branch in BRANCHES else "full"

for name, args in BRANCHES[branch]:
    findings.append(run_tool(name, args))
```

- → Constrain the label in the schema. A free string will invent branches.
- → An unknown branch must fall back to one you wrote — never to nothing.

⚠ Stop when you cannot name the branches

- A new kind of request is a **new branch**: written, reviewed, and tested like any other code.
- A router cannot change its mind halfway. The logs branch will not go and check the reactor.
- Move right when the next step depends on what the last step returned.
- That is ReAct, and it costs a model decision on every turn.



ReAct

Reason → Act → Observe, and the guards it needs

🔧 ReAct - Reason and Act

- The model must say **why** before it acts.
- That reasoning is the trace that explains itself.
- It is also a steering wheel: you can prompt against the reasoning, not just the actions.

Make the reasoning structural

```
class ReActStep(BaseModel):
    thought: str                      # required — cannot come back empty
    action: Literal["check_system", "search_logs",
                    "calculator", "final_answer"]
    action_input: str

response = client.responses.parse(
    model=MODEL, reasoning={"effort": "none"},
    text_format=ReActStep, input=messages,
)
step = response.output_parsed
```

- → The schema, not the prompt, is what guarantees a thought on every turn
- → `final_answer` is just another action — one clean exit from the loop

The loop, with its guards

```
seen = set()
for _ in range(max_steps):              # guard 1: step budget
    step = decide(messages, client)

    if step.action == "final_answer":
        return step.action_input

    sig = (step.action, step.action_input)
    if sig in seen:                     # guard 2: no progress
        messages.append(nudge(step))
        continue
    seen.add(sig)

    obs = run_tool(step.action, step.action_input)
    messages += [assistant(step), user(f"Observation: {obs}")]

return force_an_answer(messages)        # guard 3: never return None
```

- → Without guard 2 a stuck model spends the whole budget on one tool
- → Without guard 3 a timeout returns nothing at all, which is worse than a partial answer

Three guards

- **Step budget** — bounds the worst case. Without it, cost is unbounded.
- **No-progress detection** — a repeated call returns an observation it has already seen.
- **Forced finalisation** — out of budget means answer with what you have, not return `None`.



Demo — ReAct and its guards

Switch to terminal: python demo/02_react.py



Plan-and-execute

Decide once, then carry it out

Plan, execute, re-plan

```
def plan_and_execute(goal, client, max_replans=1):
    plan = generate_plan(goal, client)      # one model decision
    completed = []

    for i, step in enumerate(plan):
        try:
            step.result = execute_step(step, client)
            step.status = "done"
        except Exception as e:
            step.status = "failed"
            if max_replans > 0:                # re-plan what is left
                plan = completed + revise_plan(plan[i+1:], completed, e, client)
                max_replans -= 1
        completed.append(step)

    return synthesise(goal, completed, client)
```

- Cap `max_replans` — re-planning on every failure is an infinite loop with extra steps
- The executor for one step can itself be a small ReAct loop

The real payoff: the plan is data

- A ReAct loop lives in a call stack. A plan is a **list of objects**.
- You can **show it to a user** before anything runs — Excellent for UX.
- You can **store it**, diff intended against actual, and replay it in tests.
- You can **schedule it**
- You can **stop in the middle of it**



Demo — plan-and-execute

Switch to terminal: python demo/03_plan_and_execute.py



A plan is a graph, not a list

The thing ReAct structurally cannot do

Why ReAct is always serial

- ReAct picks its next action **after reading the last observation**.
- So every step waits on the one before it — even when the work is independent.
- Checking four ship systems has no ordering constraint, but ReAct checks them one at a time.
- A plan written up front can **declare** what depends on what.
- One extra field on the schema turns the list into a schedule.

depends_on, and the waves it implies

```
class PlannedStep(BaseModel):
    number: int
    description: str
    depends_on: list[int]        # empty = nothing is blocking this

def dependency_levels(steps):
    """Group into waves. Everything in a wave can run at once."""
    remaining, done, levels = {s.number: s for s in steps}, set(), []
    while remaining:
        ready = [s for s in remaining.values()
                 if all(d in done or d not in remaining for d in s.depends_on)]
        if not ready:                      # a cycle — do not hang
            levels.append(list(remaining.values()))
            break
        levels.append(ready)
        for s in ready:
            done.add(s.number); del remaining[s.number]
    return levels
```

- → Tell the planner not to invent dependencies — a false one costs you the speed-up
- → Always handle the cycle case: a confused plan should still run, not deadlock

Run each wave concurrently

```
async def run_parallel(levels, client):
    results = {}
    for wave in levels:
        completed = await asyncio.gather(
            *(execute_step(step, results, client) for step in wave)
        )
        for step, result in zip(wave, completed, strict=True):
            results[step.number] = result
    return results

# inside execute_step, keep blocking tools off the event loop:
result = await asyncio.to_thread(run_tool, call.name, call.arguments)
```

- → Six steps in two waves finish in the time of two steps, not six
- → A blocking tool called directly would serialise the whole wave

ReAct vs plan-and-execute, concretely

ReAct	Plan-and-execute
Serial by construction	Independent steps run concurrently
Nothing to show until it finishes	Show the plan before running it
Dies with the process	Checkpoint to disk, resume later
Cannot pause for a human	Block on approval mid-plan
Adapts the instant something surprises it	Only adapts when a step fails



Demo — parallel plans

Switch to terminal: python demo/04_parallel_plan.py



Pause, approve, resume

What you get for free once the plan is on disk

Code decides what is dangerous

- Let the **code** flag which tools need approval, from a fixed allowlist.
- Gate on the irreversible: money, messages, writes to a permanent record.
- Checkpoint after every step, so a crash costs one step rather than the whole run.
- Resuming must be **idempotent** — the guard is the status field, and you must test it.

The approval gate

```
NEEDS_APPROVAL = {"file_report", "book_flight", "send_email"}

def run_plan(state):
    approved = set(state.get("approved", []))

    for step in state["steps"]:
        if step["status"] == "done":
            continue                          # resume: never repeat a side effect

        if step["requires_approval"] and step["number"] not in approved:
            step["status"] = "awaiting_approval"
            save_state(state)                 # checkpoint, then stop
            return "awaiting_approval"

        step["result"] = run_tool(step["tool"], step["args"])
        step["status"] = "done"
        save_state(state)                     # checkpoint after every step
    return "complete"
```

- → Skipping completed steps is what makes resuming safe to call repeatedly
- → Save after every step — a crash should cost you one step, not the run



Demo — approval and resume

Switch to terminal: python demo/05_approval_resume.py



Putting it all together

Summary and exercises

Summary

Push control flow as far left as the task allows

Pipeline, then router, then plan, then ReAct. Move right only when the task breaks.

Make the reasoning structural, not hopeful

A required `thought` field always arrives. One read off `output_text` usually does not.

Every loop needs a budget, a repeat check, and a forced exit

Unbounded cost, a model going in circles, and returning None are all production outages.

The plan is data — that is the whole point

Show it, schedule it in waves, checkpoint it, and pause it for a human.

Exercises — workflows in action

01 — ReAct agent: a visible thought, plus a step budget and a repeat guard

02 — Plan-and-execute: planner and executor, with re-planning on failure

03 — Parallel and approval: depends_on waves, checkpointing, and a human gate

04 — Holiday Planner app: the full web app with plan visualisation



Module 8 — Complete

Next: multi-agent systems